



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 781 5926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 3

Team No.: 4

Project Title: Multi-Agent AI Execution and Resource Management System

Team Members

Roll Number	Student Name	Signature
2520030575	M. LIKHITHA	Likhitha
2520030198	K. VAISHNAVI	Vaishnavi

1. Abstract

The Multi-Agent AI Execution and Resource Management System is a C-based operating system project designed to provide a secure, controlled, and efficient environment for multiple autonomous AI agents. The system represents different AI agents as separate processes responsible for tasks such as code generation, program execution, code review, and application testing.

The agents communicate with each other through Named Pipes (FIFOs), enabling the exchange of structured task and execution information. A restricted workspace is provided to ensure that agents can access only authorized files and directories. Before performing any file operation, the system validates the agent's access permissions to prevent unauthorized modifications.

The system continuously monitors the CPU and memory usage of each agent. If an agent exceeds the predefined resource limits, signals are used to control, suspend, or terminate the process. Synchronization mechanisms are implemented to prevent conflicts when multiple agents access or modify shared files simultaneously.

For reliability and accountability, the system automatically maintains activity logs and execution results and creates backup copies of modified files. Thus, the project demonstrates key operating system concepts such as process management, inter-process communication, file access control, resource management, signals, synchronization, logging, and backup/recovery in a practical multi-agent AI environment.

2. Problem Statement

Multiple AI agents need to perform tasks such as code generation, execution, testing, and review at the same time. Without proper control, they may access unauthorized files, consume excessive CPU or memory, or cause conflicts while working on shared files.

Therefore, this project aims to develop a C-based multi-agent execution system that provides secure communication, restricted file access, resource monitoring, process control, file synchronization, activity logging, and backup of modified files.

3. Objectives

1. To create separate processes for different AI agents.
2. To enable agent communication using Named Pipes (FIFOs).
3. To provide a restricted workspace with controlled file access.
4. To monitor and manage CPU and memory usage of agents.
5. To use signals to control agents exceeding resource limits.
6. To ensure synchronization while accessing shared files.
7. To maintain activity logs and execution results.
8. To create backup copies of modified files for data recovery.

4. Proposed Methodology

- 1. Create AI Agent Processes:** Different AI agents are created as separate processes for Code Generation, Execution, Code Review, and Testing.
- 2. Agent Communication:** Agents communicate and exchange task information using Named Pipes (FIFOs).
- 3. Secure Workspace Access:** Each agent operates inside a restricted workspace, and file access is validated before performing any operation.
- 4. Execute & Monitor Resources:** The assigned task is executed while continuously monitoring the agent's CPU and memory usage.
- 5. Resource Control:** If an agent exceeds the allowed resource limits, Linux signals are used to control or terminate the process.
- 6. Synchronization & Collaboration:** File synchronization mechanisms prevent conflicts when multiple agents access or modify shared files.
- 7. Logging & Backup:** Agent activities and execution results are recorded, and **backup copies** of modified files are created.

Overall Flow:

Agent Creation → FIFO Communication → File Validation → Task Execution → Resource Monitoring → Synchronization → Logging & Backup

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Management (fork(), wait())	Create and manage separate AI agent processes
Named Pipes (FIFOs)	Enable communication between AI agents
File I/O (open(), read(), write(), close())	Perform controlled file operations and maintain logs
File Access Control (access())	Validate authorized file access
Signals (signal(), kill())	Control or terminate agents exceeding resource limits
Resource Monitoring (/proc, getrusage())	Monitor CPU and memory usage of agents
Synchronization (flock())	Prevent conflicts and race conditions on shared files
Backup & Recovery	Maintain backup copies of modified workspace files

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030575	M. LIKHITHA	Implement AI agent process creation using fork () and process management. • Implement Named Pipes (FIFOs) for communication between agents. • Handle task execution and inter-process coordination. • Implement signals for agent control and termination.
2520030198	K. VAISHNAVI	Implement the restricted workspace and file access validation. • Implement CPU and memory resource monitoring. • Handle file synchronization and shared-file protection. • Implement activity logging and file backup/recovery.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating system environment
C / C++	Develop the multi-agent execution system
Visual Studio Code	Write and manage source code
Git / GitHub	Version control and project collaboration
Bash Terminal	Execute and test linux commands and programs
GCC	Compile and run C programs

8. Expected Outcome

The proposed system is expected to provide a secure and controlled environment where multiple AI agents can execute tasks independently and communicate using Named Pipes (FIFOs). It will restrict unauthorized file access, monitor CPU and memory usage, and control agents that exceed resource limits. The system will also ensure conflict-free file sharing through synchronization, maintain activity logs, and create backups of modified files, resulting in a reliable, secure, and resource-efficient multi-agent execution system.

9. GitHub Repository

GitHub Repository Link: <https://github.com/likhithamikkilineni/Multi-Agent-AI-Execution-and-Resource-Management-System-OSSP>

Repository Created: Yes

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: K. Hema

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____